

ICML 2024: Why Deep Networks Memorize Their Data, Finally Explained

A Purdue University team ties memorization, input loss curvature, and differential privacy into a single triangle of provable bounds

When we train a deep neural network, we want it to learn patterns, not to file away individual training examples verbatim. Yet a large body of work shows that deep networks do memorize part of their training data, especially the rare, hard, and noisy examples. That memorization is entangled with generalization and noisy learning, and it is also a privacy problem: if a model has memorized someone's data, an attacker may be able to pull it back out.

The trouble is measuring which examples a model has actually memorized. Feldman's stability-based memorization score gives a rigorous definition, but it is brutally expensive: scoring a single example means training hundreds or thousands of models on datasets that do and do not contain it. So researchers went looking for a cheaper stand-in, and the most promising one is input loss curvature, the trace of the Hessian of the loss with respect to the input. It costs roughly three orders of magnitude less to compute than the memorization score.

Until now, using curvature as a proxy for memorization was purely empirical. The two quantities rose and fell together in experiments, but nobody could say why, and nobody knew how either related to differential privacy. This paper, from Purdue University and presented at ICML 2024, supplies the missing theory. It ties memorization, input loss curvature, and differential privacy together with a set of rigorous upper bounds that close into a triangle.

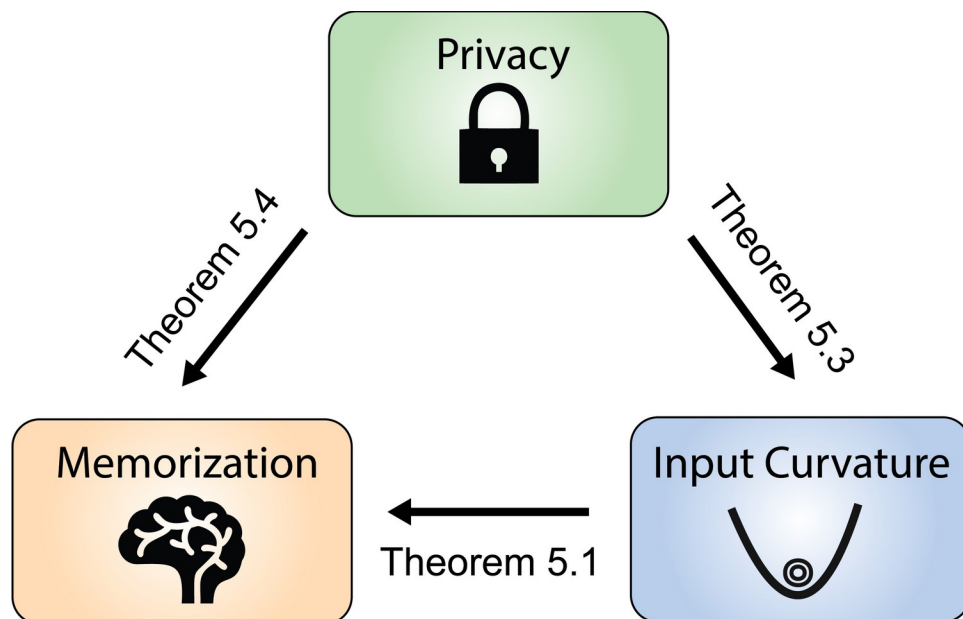


Figure 1. The paper's framework. Theorem 5.1 bounds memorization by input loss curvature, Theorem 5.3 bounds average curvature by the differential-privacy parameter, and Theorem 5.4 chains the two to bound memorization directly by privacy. The three edges close into a triangle.

Paper and code: the arXiv paper is at <https://arxiv.org/abs/2402.18726> .

Why a cheap proxy still needs a theory

The usual tool for counterfactual questions in deep learning, the kind that ask what happens if we remove one example, is the influence function. But influence functions lean on a strong assumption: that the loss Hessian is convex and positive-definite. For real deep networks, that condition essentially never holds. Input loss curvature, by contrast, only needs a second-order expansion of the loss and makes no assumption about Hessian convexity or definiteness, which makes it a better fit for analyzing deep networks.

But cheap and convenient is not the same as correct. Without a proof that curvature actually controls memorization, swapping in curvature for the memorization score is just a lucky empirical habit. The question the paper sets out to answer is exactly this: can the link between curvature and memorization be written as a checkable mathematical bound?

What curvature is actually measuring

Before the theory, it helps to build some intuition. If you rank the training set by input loss curvature, a clean stratification appears. Low-curvature examples are prototypical members of their class, while high-curvature examples are the rare, difficult, more-likely-memorized outliers. That is precisely the behavior the memorization score captures, and it is the intuitive reason curvature can serve as a proxy at all.



Figure 2. ImageNet training images ranked by input loss curvature from a single ResNet18, one class per row. The ten left columns are the lowest-curvature samples, canonical in pose and background; the ten right columns are the highest-curvature samples, with unusual framing and clutter that look more like memorized instances.

Three bounds, one for each edge of the triangle

The theory starts from a second-order Taylor expansion of the loss, using the Nesterov-Polyak bound to bring in the input Hessian. The key move is to choose a zero-mean perturbation so that the first-order terms vanish under expectation, leaving an upper bound on memorization written in terms of input loss curvature plus a data-independent constant that bundles up stability, generalization, and model-bias terms. That is Theorem 5.1: curvature controls memorization.

A parallel derivation, combined with the definition of differential privacy, yields a new result, Theorem 5.3: a stronger privacy guarantee, meaning a smaller privacy budget ϵ , forces lower average input loss curvature. Chain the two edges and you get Theorem 5.4, which bounds memorization directly by privacy. None of these steps assumes Hessian convexity or positive-definiteness, so the results carry over directly to the deep networks we actually train.

To make curvature practical to measure, the authors estimate the trace of the Hessian with Hutchinson's trace estimator, configured with a finite-difference step of $h = 1e-3$ and $n = 10$ Rademacher probe vectors, sidestepping any need to form the full Hessian.

Edge one: memorization rises linearly with curvature

A theory has to earn its keep on real data. The authors reuse Feldman & Zhang's (2020) precomputed memorization scores and released model ensembles: 1000 Small Inception models on CIFAR100 and 100 ResNet50 models on ImageNet, averaging each example's memorization score and curvature across the ensemble.

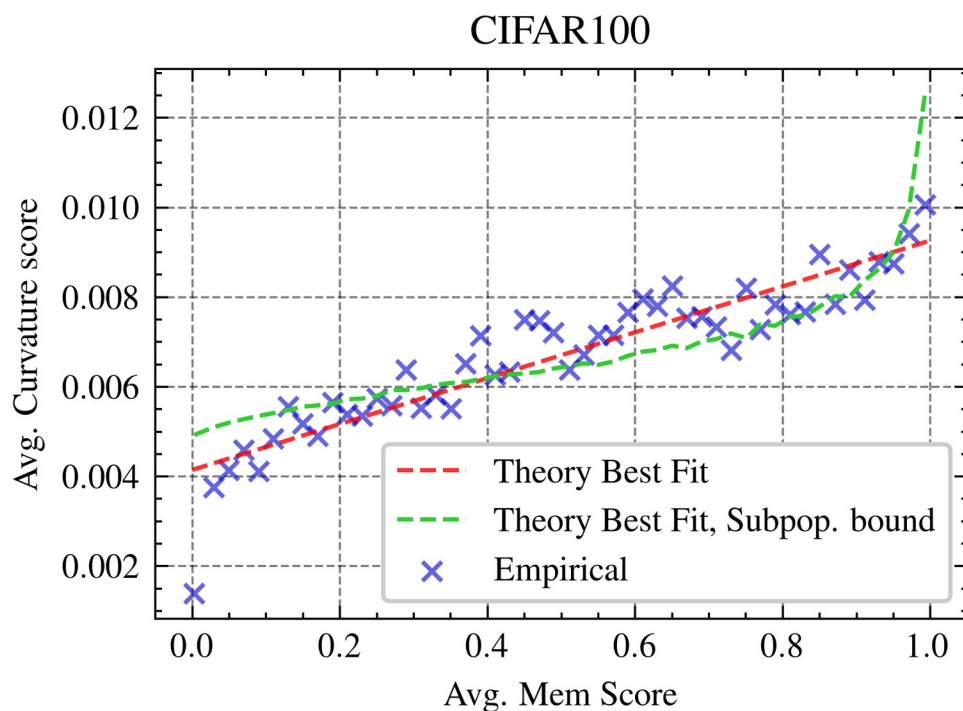


Figure 4. At the end of training on CIFAR100, average memorization score (x-axis) against average curvature score (y-axis), averaged over 1000 Small Inception models. Blue crosses are empirical data, the red dashed

line is the theory best fit, and the green dashed line is the sub-population bound. The points hug the predicted curve, confirming Theorem 5.1.

The scatter tracks the predicted curve almost exactly: the higher an example's memorization score, the larger its input loss curvature, in a clean, monotone, near-linear relationship. The paper notes the trend is especially crisp on ImageNet. This suggests curvature is not merely correlated with memorization by accident, but bounds it in the way the theorem predicts.

Edge two: stronger privacy pushes curvature down

For the privacy edge, the authors train ResNet18 with DP-SGD, sweeping the privacy budget ϵ over 1, 10, 20, 30, 40, and 50 at a fixed $\delta = 1e-5$, averaging over 10 random seeds per budget, and measuring the resulting input loss curvature.

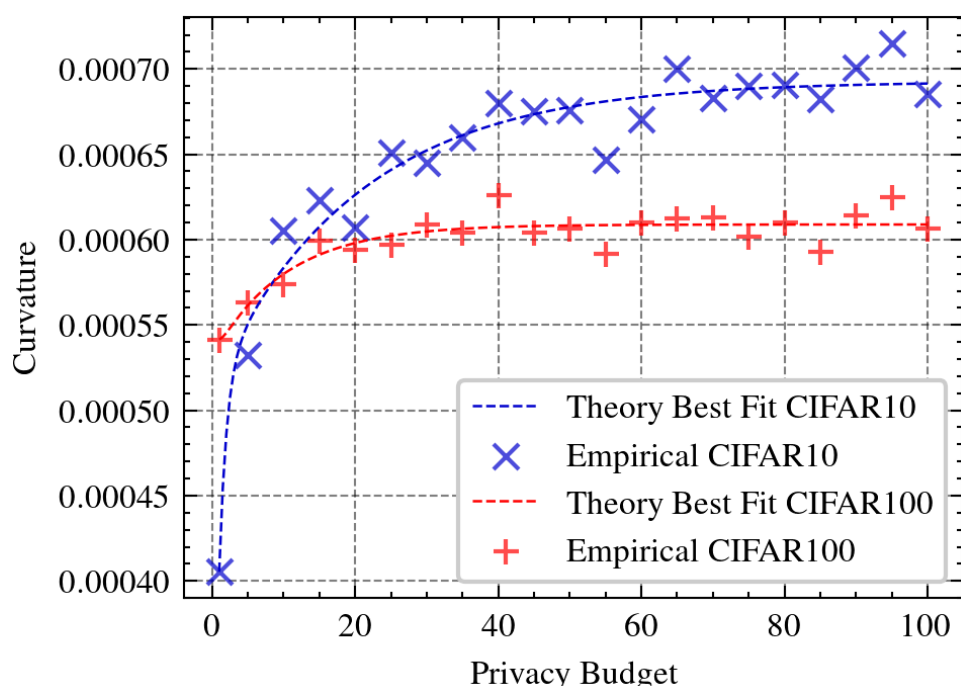


Figure 7. Privacy budget (x-axis, smaller means stronger privacy) against input loss curvature (y-axis) for CIFAR10 and CIFAR100. The dashed best-fit curve is the shape predicted by Theorem 5.3, and the empirical points land on it: a tighter budget yields lower curvature.

As the privacy budget tightens (smaller ϵ), input loss curvature falls, and the shape of that decline matches the best-fit curve the theory predicts. In other words, the differential-privacy mechanism drives input loss curvature down along exactly the trajectory Theorem 5.3 predicts.

Edge three: privacy caps memorization

The last edge connects privacy and memorization directly. On CIFAR100, the authors watch how memorization scores move with the budget once differential privacy is applied, and compare against the upper bound Theorem 5.4 provides.

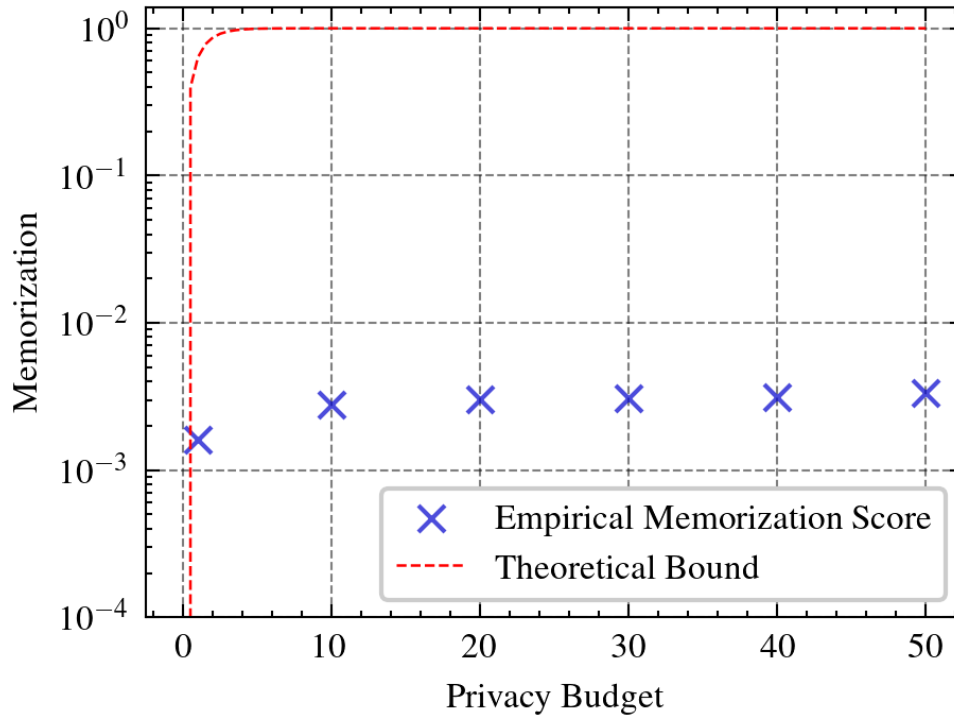


Figure 8. Differential-privacy budget (x-axis) against memorization (y-axis, log scale) on CIFAR100. Blue crosses are empirical memorization scores; the red dashed line is the Theorem 5.4 upper bound. The empirical values stay far below the bound, showing privacy really does cap memorization.

As the budget loosens, the memorization of the affected examples, particularly the most-memorized ones, rises as expected, but stays comfortably beneath the theoretical bound throughout. With that, all three edges close: curvature bounds memorization, privacy bounds curvature, and so privacy bounds memorization too.

The numbers at a glance

Table 1. Key settings and numbers from the paper.

Item	Setting / value
Curvature cost vs. memorization score	~3 orders of magnitude cheaper (~1/1000)
Memorization ensembles	CIFAR100: 1000 Small Inception; ImageNet: 100 ResNet50
Privacy-experiment model	ResNet18 + DP-SGD
Privacy budget ϵ sweep	1, 10, 20, 30, 40, 50 ($\delta = 1e-5$)
Curvature estimator	Hutchinson trace, $h = 1e-3$, $n = 10$ Rademacher vectors
Three theorems	Theorem 5.1 / 5.3 / 5.4

Why it matters, and where it stops

The contribution is that a relationship which used to be an empirical coincidence, curvature standing in for memorization, now rests on firm theory. Curvature is provably an upper bound on memorization, curvature is itself capped by the differential-privacy parameter, and so privacy bounds memorization as well. Because the framework needs no Hessian-convexity

assumptions, it applies directly to real deep networks, and its predictions hold up on CIFAR and ImageNet.

For a practitioner, the most immediate payoff is a screening tool. Because high-curvature examples are the rare, hard, likely-memorized ones (Figure 2), and curvature is cheap to compute, a single trained model plus Hutchinson's estimator can surface the training points most at risk of being memorized, without the thousand-model ensemble Feldman's score demands. The theory is what licenses reading curvature this way, rather than treating the alignment as a lucky correlation.

The privacy edge adds a second lever. Since a tighter privacy budget provably lowers both curvature and memorization, curvature also becomes a cheap diagnostic for how much a differentially private training run is actually suppressing memorization, a quantity that is otherwise expensive to audit directly.

The boundaries are worth stating plainly: these are upper-bound results resting on assumptions of error stability, generalization, model bias, and dataset adjacency, together with bounded loss, and the experiments center on image classification with standard vision architectures. Still, the practical message is clear. Using cheap curvature in place of expensive memorization scores now has a theoretical license, and privacy mechanisms reduce leakage precisely because they provably drive down both curvature and memorization at once.